Technical Specification: HLS Stream Detection Architecture

Overview

This document outlines the implementation details and code changes required to intercept and detect HLS streaming links (`.m3u8`, `.json` manifests) within the browser extension context.

---

1. Extension Manifest Configuration (`manifest.json`)

```
{
  "manifest_version": 3,
  "name": "HLS Stream Finder",
  "version": "1.0",
  "permissions": ["storage", "downloads",
        "declarativeNetRequest"],
  "host_permissions": ["<all_urls>"],
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content.js"],
    "run_at": "document_start"
  }],
  "web_accessible_resources": [{
    "resources": ["injected.js"],
    "matches": ["<all_urls>"]
  }],
  "background": { "service_worker": "background.js" },
  "action": { "default_popup": "popup.html" }
}
```

---

2. Main World Network Sniffer (`injected.js`)

```
(function() {
  const originalFetch = window.fetch;
  window.fetch = async function(...args) {
    const response = await originalFetch.apply(this, args);
    try { checkAndReport(response.url); } catch (e) {}
    return response;
  };
  const originalOpen = XMLHttpRequest.prototype.open;
  XMLHttpRequest.prototype.open = function(method, url, ...args) {
    checkAndReport(url);
    return originalOpen.apply(this, [method, url, ...args]);
  };
  function checkAndReport(url) {
    if (typeof url === "string" && (url.includes(".m3u8") ||
        url.includes(".json") || url.includes("playlist"))) {
      document.dispatchEvent(new
        CustomEvent("HLS_STREAM_DETECTED_EVENT", { detail: { url:
        url } }));
    }
  }
})();
```

---

3. Isolated World Content Bridge (`content.js`)

```
(function() {
  const script = document.createElement("script");
  script.src = chrome.runtime.getURL("injected.js");
  script.onload = () => script.remove();
  (document.head || document.documentElement).appendChild(script);
  document.addEventListener("HLS_STREAM_DETECTED_EVENT", (event)
        => {
    if (event.detail && event.detail.url) {
      chrome.runtime.sendMessage({ action: "STREAM_DETECTED",
        url: event.detail.url });
    }
  });
})();
```

---

4. Background State Manager (`background.js`)

```
const tabStreams = {};
chrome.tabs.onUpdated.addListener((tabId, changeInfo) => {
  if (changeInfo.status === "loading") {
    delete tabStreams[tabId];
    chrome.action.setBadgeText({ text: "", tabId: tabId });
  }
});
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  if (request.action === "STREAM_DETECTED") {
    const tabId = sender.tab ? sender.tab.id : null;
    if (tabId) {
      if (!tabStreams[tabId]) tabStreams[tabId] = new Set();
      tabStreams[tabId].add(request.url);
      chrome.storage.local.set({ currentStream: request.url,
        ["tab_" + tabId]: Array.from(tabStreams[tabId]) });
      chrome.action.setBadgeText({ text: "1", tabId: tabId });
      chrome.action.setBadgeBackgroundColor({ color: "#28a745" });
    }
    sendResponse({ status: "success" });
  } else if (request.action === "GET_STREAMS") {
    sendResponse({ streams: tabStreams[request.tabId] ?
        Array.from(tabStreams[request.tabId]) : [] });
  }
  return true;
});
```